ZK-STARK FRI 프로토콜의 GPU/CPU 가속 및 메모리 계층 병목 분석

Introduction

1.1 Background

블록체인 확장성 솔루션 중 하나인 layer2와 privacy 보호 기술의 핵심으로 zero-knowldge proof, 특히 양자 컴퓨팅으로부터도 안전하고 trusted setup이 필요없는 zk-stark[1]가 주목받고 있다. 그러나 zk-stark는 방대한 연산량과 메모리 접근을 요구하며, 증명 생성 시간이 서비스 적용의 가장 큰 걸림돌이 되고 있다. zk-stark의 핵심 구성 요소인 FRI protocol[2]은 대규모 다항식 연산과 merkle tree 구성을 포함하여, 시스템의 computer power와 memory bandwidth를 동시에 요구하는 특성을 가진다.

1.2 Problem Statement

기존의 single-thread CPU 환경에서는 다항식의 차수가 커질수록 시간이 기하급수적으로 증가하여 실시간 증명 생성이 불가능하다. (증명을 위해 만든 다항식의 차수는 2^{20}~2^{23}) FRI protocol은 크게 두 가지 병목 구간을 가진다

1) NTT 및 domain folding

2) Merkle tree 구성

이러한 작업들을 빠른 시간안에 완료하기 위해서는 GPU와 Data Parallelism을 활용한 가속이 필수적이며, CPU의 멀티 코어와 GPU의 대규모 병렬 아키텍처 간의 성능 특성을 비교 분석할 필요가 있다.

1.3 Objectives

본 프로젝트는 다항식의 규모에 따른 FRI protocol을 Baseline(Single thread), OpenMP(Multi-thread), CUDA의 세 가지 환경에서 구현하고 각각의 성능을 정량적으로 분석하는 것을 목표로 한다. 또한, product 수준으로 사용할 수 있도록 fiat-shamir 변환을 적용한 non-interactive 증명 시스템을 제공한다.

1.4 Scope & Environment

GPU 가속에 집중하기 위해 경량화된 hash 함수를 사용하며, memory hierarchy, data-dependency 및 CPU-GPU 간의 데이터 이동 비용에 집중하여 성능을 평가하였다.

Theoretical Background

1) NTT (Number Theoretic Transform)

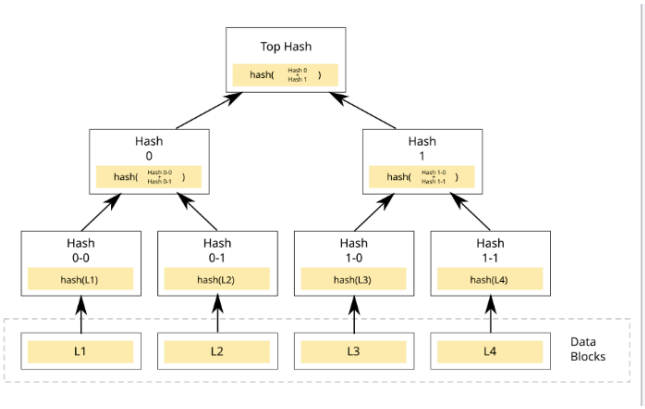
NTT란 FFT(Fast Fourier Transform)을 Finite Field에서 진행한 버전이다. FFT란 임의의 N차 다항식을 N개의 점에서 평가하는데, $O(N^2)$이 아닌 $O(NlogN)$만에 처리하는 방식이다. 구체적인 동작 과정은 해당 사이트를 참고하길 바란다.[3]

간단히 설명하자면, 다항식 $A(x) = a_0x^0 + a_1x^1 + \ldots + a_{n-1}x^{n-1}$이라 하자.
$A_0(x) = a_0x^0 + a_2x + \ldots + a_{n-2}x^{n/2-1}$, $A_1(x) = a_1x^0 + a_3x + \ldots + a_{n-1}x^{n/2-1}$
로 정의하면 $A(x) = A_0(x^2) + xA_1(x^2)$ 이 성립한다. 이 때, x를 적절히 선택하면($x^n = 1$), n개의 점에서 A의 값을 $O(nlogn)$만에 평가할 수 있다.

본 연구에서는 메모리 사용량을 최소화하기 위해 In-place 알고리즘을 선택하였으며, 이를 위해 전처리 단계로 Bit-Reversal Permutation을 수행했다. Bit-Reversal Permutation이란 divide-and-conquer에서 divide를 한 번에 끝내는 작업이다.

2) Merkle Tree

Merkle Tree란 데이터 integrity를 보장하기 위해 사용하는 기법이다. 각 leaf에 실제 데이터를 넣고, 각 leaf끼리 hash 함수를 적용해 최종 root를 만드는 과정이다.



[4]

본 연구에서는 약 2^{20}개의 데이터를 바탕으로 merkle tree를 만들어야 했기 때문에, 중요한 병목지점이었다. 따라서 이것 또한 openmp와 cuda를 통해서 최적화했다.

Algorithm

NTT와 merkle tree를 빠르게 실행하기 위해 고려한 포인트는 다음과 같다.

1) CPU와 GPU간의 데이터 통신

먼저 CPU와 GPU간의 데이터 통신을 최소화하기 위해 다항식의 계수과 x값들을 처음에 GPU에 보낸 후에는 merkle tree 생성까지 한 후 CPU로 가져왔다. 즉, 하나의 증명을 생성하는데 한번씩만 데이터를 이동시키면 된다.

2) domain generate

FFT에서 x값들은 1, w, w^2, ... , w^{n-1} 같은 특성을 가지고 있다. 따라서 GPU를 통해 각각의 요소를 만들었다. 또한, 단순히 n-1번 곱하는 것이 아닌 bit로 표현해서 $logn$만큼 곱하도록 해 속도를 향상시켰다. 예를 들어, w^6이면, 110 로 표현할 수 있고, $(1 * w^4) * (1 * w^2) * (0 * w^0)$이 되어 $logn$만에 할 수 있다.

3) global and shared memory

In-place 알고리즘을 이용하면 모든 데이터가 global 영역에 저장되기 때문에 단순히 global에 계속 쓰면 시간이 오래 걸린다. 따라서 share memory를 쓰는 단계와 global memory를 쓰는 단계를 나누어서 속도를 향상시켰다.

구체적인 코드는 해당 github에 있다.[5]

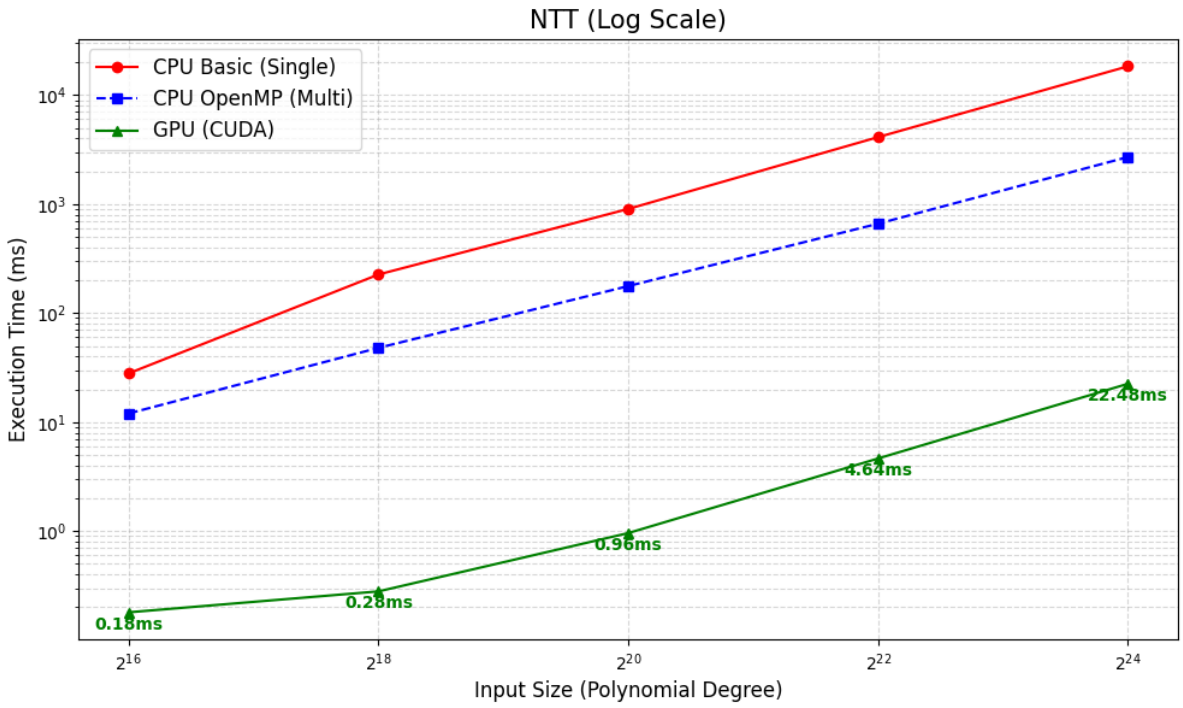
Environment

실험은 성균관대학교 인의예지 서버 중 ji 서버에서 실행하였고, 사용한 gpu는 NVIDIA TITAN V이다.

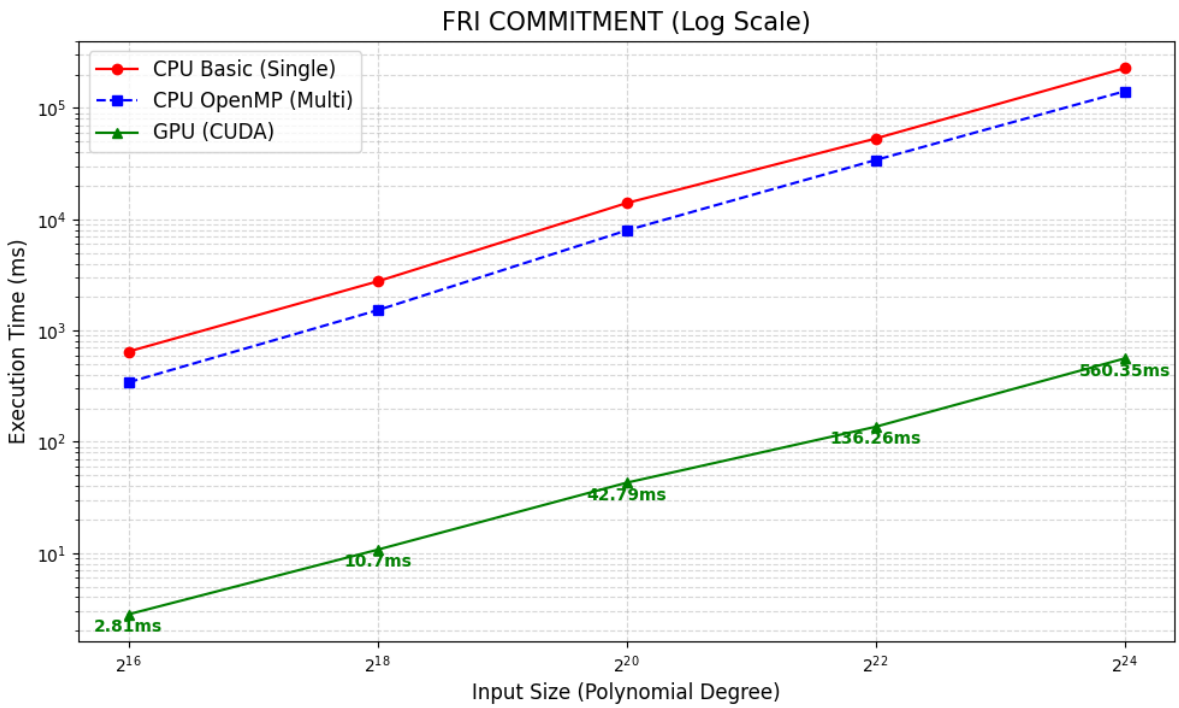
Result

다양한 다항식 차수에 따른 NTT와 FRI 결과를 측정하였다. GPU와 CPU 간의 결과 차이가 너무 크기 때문에 log scale로 변환하였다.

먼저 다항식 차수에 따른 NTT 결과이다.



다음은 FRI commitment에 대한 결과이다.



NTT에서는 데이터의 크기가 커질수록 GPU 가속이 극적으로 나타남을 확인하였다. 특히 가장 큰 입력 크기인 $N = 2^{24}$를 기준으로 보았을 때 성능 차이는 다음과 같다

- CPU (Single Thread): 약 18365ms (18.3초) 소요
- CPU (OpenMP): 약 2704ms (2.7초) 소요 (Single 대비 약 6.8배 향상)
- GPU (CUDA): 약 22.48ms (0.02초) 소요 (Single 대비 약 816배 향상)

CPU 환경에서는 OpenMP를 통한 multi threading이 약 6~7배 성능 향상을 보였으나, compute bound인 NTT 알고리즘의 특성상 수천 개의 core를 활용하는 GPU의 병렬 처리 성능이 압도적이었다.

FRI에서는 NTT와 마찬가지로 GPU가 가장 우수한 성능을 보였으나, NTT와 다른 양상이 관찰되었다.

- CPU (Single Thread): 약 227556ms (227초) 소요
- CPU (OpenMP): 약 141344ms (141초) 소요 (Single 대비 약 1.6배 향상)
- GPU (CUDA): 약 560.35ms (0.02초) 소요 (Single 대비 약 406배 향상)

NTT와 달리 FRI commitment 단계는 대량의 hash 연산을 수행하며 빈번한 메모리 접근을 요구한다. 이로 인해 CPU 환경에서는 계산 속도보다 memory bandwidth가 병목이 되어, multi thread를 사용하더라도 성능 향상 폭이 제한적이었다. 반면 GPU는 이러한 병목을 해소하며 약 400배 이상의 성능을 유지하였다. NTT보다는 성능이 떨어진 이유는 merkle tree를 구성할 때, 동기화 비용 및 global memory에 자주 접근해야 했기 때문이다.

Conclusion

실험 결과, 입력 크기가 2^{24}에 달하는 대규모 연산 환경에서 CPU 기반 처리는 분 단위의 시간이 소요되어 실시간 증명 생성에 부적합함을 확인하였다. 반면, GPU 가속을 적용할 경우 이를 ms 단위로 단축할 수 있었다.

ethereum의 블록 생성 시간이 15초인 것을 감안한다면, 블록 생성 시간 내에 안정적으로 증명을 생성하기 위해 GPU를 사용하는 것은 선택이 아닌 필수로 보인다.

Reference

1: zk-stark: Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018. Available at https://eprint.iacr.org/2018/046

2: Fri-protocol: Eli Ben-Sasson, Iddo Bentov, Ynon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In Proceedings of the 45th International Colloquium on Automata, Languages, and Programming (ICALP), 2018.

3: FFT: https://cp-algorithms.com/algebra/fft.html

4: https://en.wikipedia.org/wiki/Merkle_tree

5: https://github.com/taejuk/fft_cuda/tree/main/fri